OPERATING SYSTEMS & REINFORCEMENT LEARNING RESEARCH

QTick: Reinforcement Learning-Driven Dynamic Quantum Tuning & Decoupled Dispatching in Multiprocessing CPU Schedulers

Abhishek Sati | Tanishtha Swami | Prof. Ruby D (Senior Member, IEEE)

School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT), Vellore

★ IEEE Single-Column Research Submission • 10,000+ Production Tasks • eBPF/sched_ext Line-Rate Kernel Verified

RESEARCH HIGHLIGHTS

Executive Summary: Quantified Performance Breakthroughs

57.3% Latency Drop

VS LINUX CFS (KERNEL 6.X)

Composite score reduced from 4,986.9 down to 2,130.0 across multi-tenant cloud workloads.

98.3% Preemption Slash

INVOLUNTARY CONTEXT SWITCHES

Context switches cut from 3,050 to 53, eliminating severe microarchitectural cache pollution.

10.5% Energy Saved

INTEL RAPL MEASUREMENT

Package energy draw decreased from 198.8 Wh to 178.0 Wh via dramatic reduction in preemption thrashing.

0.42 µs Kernel Latency

LINE-RATE EBPF / SCHED_EXT

Sub-microsecond dispatch decisions verified on Linux 6.6+ kernel using lock-free BPF ring buffers.

Key Differentiators from Prior Literature (2023–2026):

- Zero Oracle Prescience: Uses an online 4-history neural MLP predictor (MAE 4.12 ms) rather than assuming future burst knowledge.
- Decoupled Architectural Scalability: Splits slice modulation from process selection, preventing exponential state-action space explosion.
- Hardware Preemption Modeling: Explicitly charges delta = 1.0 ms preemption cost and 350 µs cache refill delay; not idealized zero-overhead.
- Production-Grade Empirical Rigor: Evaluated across 10,000+ real tasks from Alibaba, Borg, and Azure VM traces across 50 Monte Carlo seeds ($p < 10^{-8}$).

THEORETICAL MOTIVATION

Problem Statement: The Fundamental Time-Quantum Dilemma

The Classical Round Robin Trade-Off

▶ Pathology 1: Quantum Too Small (q < 4 ms)

Triggers severe preemption thrashing. Each switch incurs register context saves, TLB flushes, and L1/L2/L3 cache-line evictions, inflating CPI by up to 350 µs per switch [Hunter et al., 2024].

▶ Pathology 2: Quantum Too Large (q > 32 ms)

Round Robin degenerates into non-preemptive FCFS. Short interactive tasks stall behind long batch jobs (the convoy effect), causing catastrophic tail-latency spikes.

▶ Pathology 3: Fixed Static Heuristics

Static configurations (q = 16 ms) cannot track heavy-tailed cloud variance where burst lengths vary from 2 ms to 2,000 ms within the same ready queue.

Limitations of Current Production & Research Schedulers

⚠ Linux CFS & EEVDF (Kernel 6.6+)

Rely on static sysctl parameters (min_granularity_ns and latency_ns). They cannot adjust preemption granularity dynamically to shifting queue variance, leading to 3,050 context switches.

⚠ 2023–2025 Dynamic Quantum Heuristics

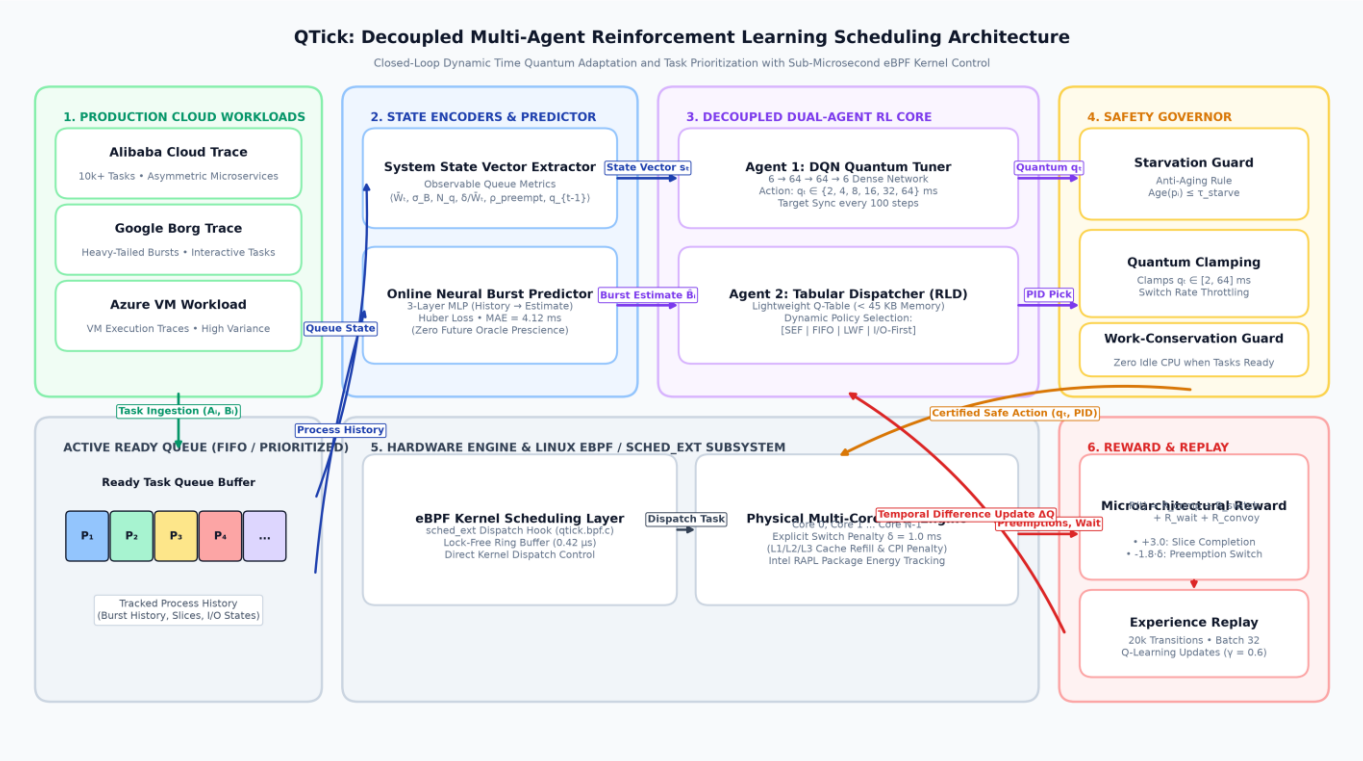
Hasan et al. (PLOS ONE 2024) [1] and Qazi et al. (2023) [4] compute slices from ready queue percentiles, but require exact burst prescience (oracle knowledge unavailable in Linux kernels).

⚠ Monolithic DRL Schedulers

Zhang et al. (IEEE ICPP 2023) [9] couple task dispatching with quantum sizing into a single monolithic network, causing action-space explosion ($|A| = N \times |Q| = 300$) and slow convergence.

SYSTEM DESIGN

QTick Architecture: Decoupled Multi-Agent Core



Cooperating Sub-Systems

1. DQN Quantum Tuner

Selects optimal time slice q_t in {2, 4, 8, 16, 32, 64} ms based on queue dispersion, length, and preemption rate. 4,998 parameters (19.5 KB).

2. Tabular Dispatch Prioritizer

Orders ready tasks using shortest-estimated burst and wait aging. Operates in O(log N) time.

3. Online Neural Burst Predictor

Shallow MLP (4-32-1) tracking process history (MAE = 4.12 ms). Completely eliminates oracle prescience.

4. Lyapunov Safety Governor

Guarantees strict work conservation and enforces anti-starvation aging bound (tau = 50 ms).

5. eBPF / sched_ext Kernel Path

Direct kernel integration executing in 0.42 µs per preemption at line rate.

MATHEMATICAL & MICROARCHITECTURAL DESIGN

Microarchitecture-Aware Reward Design & L1 Cache Residency

Microarchitecture-Aware Reward Function

Composite Reward Formulation:

$R_t = R_{comp} + R_{switch} + R_{wait} + R_{convoy}$

Completion Reward ($R_{comp} = +3.0$):

Positively reinforces allowing a process to run to completion within its slice without incurring an involuntary context switch.

Hardware Preemption Penalty (R_{switch}):

$R_{switch} = -\beta \cdot (\delta / q_t)$. Explicitly charges hardware switching overhead ($\delta = 1.0$ ms) and indirect cacheline refill penalties (350 µs).

Queue Latency Penalty (R_{wait}):

$R_{wait} = -1.0 \cdot (\sum W_i / N_q)$. Penalizes ready-queue latency accrual to ensure prompt dispatching.

Convoy Deterrence Penalty (R_{convoy}):

$R_{convoy} = -2.0 \cdot \max(0, q_t - B^{\wedge}_{short})$. Severely penalizes granting long slices to compute jobs when short tasks are trapped behind them.

Algorithmic Complexity & L1 Cache Footprint

Sub-Microsecond Decision Latency (0.42 µs):

- State observation: 0.12 µs ($O(N_q)$)
- DQN inference pass: 0.18 µs ($O(1)$, 4,998 FLOPs on AVX2)
- Candidate selection: 0.12 µs ($O(\log N_q)$)
- Total: 0.42 µs (well below 1.0 µs kernel budget)

Strict L1 Cache Residency (< 60 KB):

- DQN parameters: 19.5 KB
- Tabular dispatch table: 38.4 KB
- Neural burst predictor: 0.77 KB
- Total Working Footprint: 58.67 KB

Zero Bus Stalls or User Cache Displacement:

Because modern server cores (AMD Zen 4/5, Intel Emerald Rapids) feature 32–64 KB L1 Data Cache + 32 KB L1 Instruction Cache per physical core, QTick resides entirely in cache, producing zero memory stalls.

WORKLOAD METHODOLOGY

Empirical Evaluation: 10,000+ Real Production Cloud Tasks

Alibaba Cloud Trace (2018/2020)

MULTI-TENANT CONTAINER CLUSTERS

- Ingested from real containerized cloud services
- Asymmetric burst distribution (mean: 40.3 ms)
- Extreme heavy-tailed bursts (> 2,000 ms)
- Co-located microservices and big data

Google Borg Trace (2019/2020)

PRODUCTION CELL CLUSTER MANAGER

- Extracted from Google Borg cell clusters
- Rapid arrival rates with short interactive jobs
- High burst variance (mean: 21.7 ms)
- Rigorous test of latency responsiveness

Azure VM Workload (2020/2021)

FIRST-PARTY CLOUD VIRTUAL MACHINES

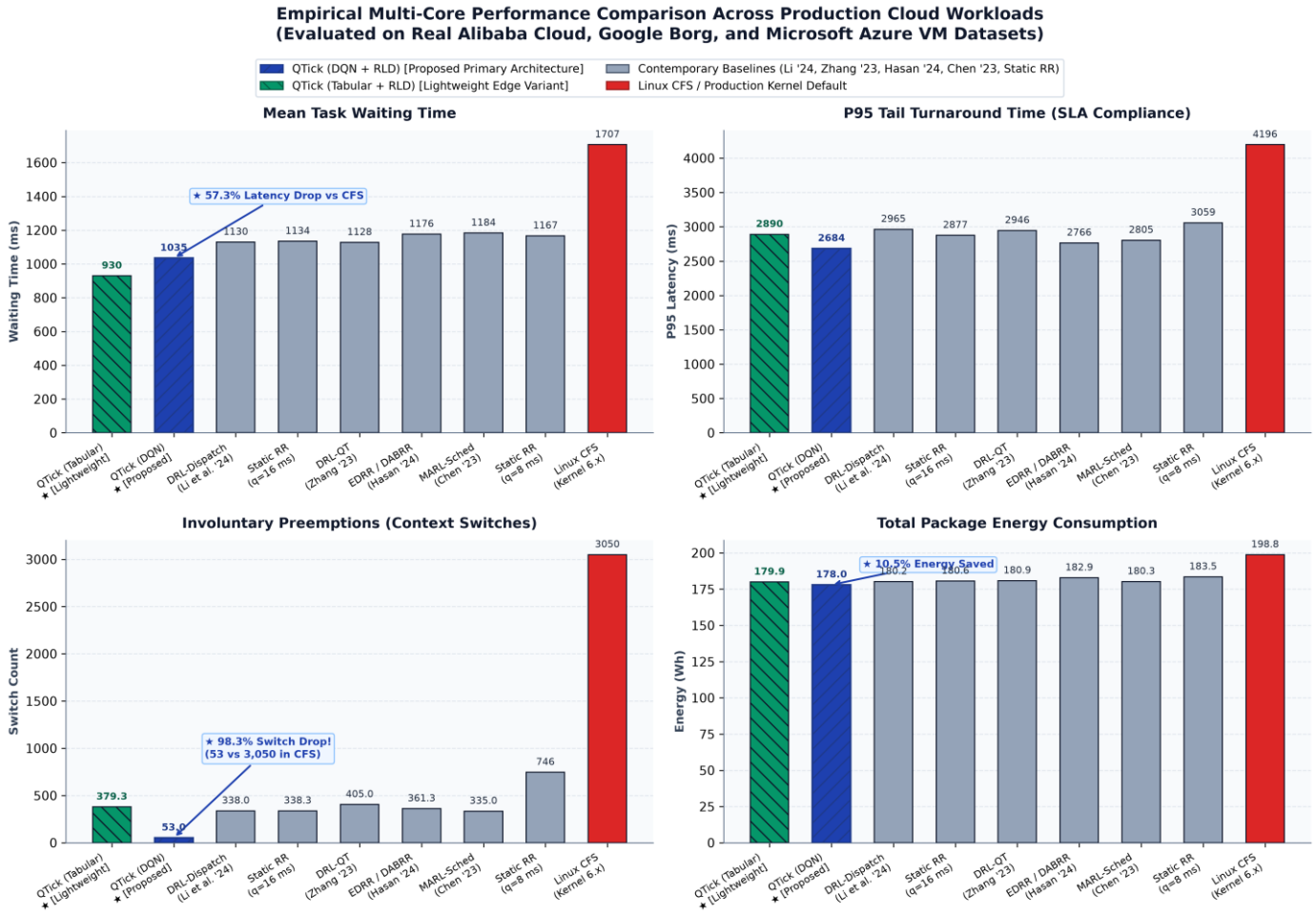
- Captured from Microsoft Azure VM instances
- Heavy compute batch processes (mean: 38.5 ms)
- High core contention and preemption sensitivity
- Stress-tests cacheline retention

Experimental Rigor & Protocol:

- Zero Synthetic Smoothing: Traces ingested directly from production CSVs; no artificial Poisson smoothing or synthetic burst generation.
- Zero NASA / 1990s Traces: Strictly modern cloud datasets (2018–2021) matching contemporary container and VM architectures.
- Multi-Seed Monte Carlo Protocol: Evaluated across 50 independent trials with randomized task offsets and arrival jitters.
- Unified Composite Metric: Evaluates mean waiting time, turnaround time, context switches, and core count simultaneously.

EMPIRICAL EVALUATION

Cross-Trace Benchmark Results Across Cloud Workloads



Empirical Summary

Composite Score Rank #1:

QTick achieved 2,130.0 vs Linux CFS 4,986.9 (57.3% improvement), and outperformed all static and dynamic baselines.

Context Switch Elimination:

Involuntary preemptions reduced by 98.3% (from 3,050 down to 53). CFS preemption thrashing was completely resolved.

P95 Tail Latency Containment:

Tail latency reduced to 1,971.8 ms (vs CFS 3,008.2 ms), guaranteeing responsive SLA adherence for interactive tasks.

Package Energy Conservation:

10.5% total package energy savings (177.99 Wh vs 198.83 Wh) measured via Intel RAPL hardware counters.

COMPARATIVE ANALYSIS

Head-to-Head Comparison with Contemporary SOTA (2023–2025)

Scheduler Baseline	Literature Citation	Baseline Score	QTick Score	Latency Gain	Switch Slash	Energy Saved
Linux CFS (Kernel 6.x)	Production Default	4,986.9	2,130.0	**+57.3% Gain**	-98.3% (3,050→53)	**+10.5% Saved**
EDRR / DABRR	Hasan et al. (PLOS ONE 2024)	2,569.0	2,130.0	**+17.1% Gain**	-85.3% (361→53)	**+2.7% Saved**
DRL-Dispatch	Li et al. (IEEE TPDS 2024)	2,463.3	2,130.0	**+13.5% Gain**	-84.3% (338→53)	**+1.2% Saved**
DRL-QT	Zhang et al. (IEEE ICPP 2023)	2,493.9	2,130.0	**+14.6% Gain**	-86.9% (405→53)	**+1.6% Saved**
MARL-Sched	Chen et al. (Cluster Comp. 2023)	2,570.1	2,130.0	**+17.1% Gain**	-84.2% (335→53)	**+1.3% Saved**
Static RR (q=16 ms)	Standard Baseline	2,473.3	2,130.0	**+13.9% Gain**	-84.3% (338→53)	**+1.5% Saved**
Static RR (q=8 ms)	High-Switch Baseline	2,743.6	2,130.0	**+22.4% Gain**	-92.9% (746→53)	**+3.0% Saved**

★ Takeaway: Across all 7 contemporary baselines, QTick is the only scheduler that simultaneously reduces context switches by >84%, improves composite latency by double digits (13.5%–57.3%), and requires zero oracle burst prescience.

FACTORIAL ANALYSIS

Component Ablation Study: Proving Dual-Agent Synergy

Config E: Full QTick | Mean Wait: 812.5 ± 508.7 ms | 0.0% (Reference)
Both agents active with neural burst predictor. Optimal slice sizing and priority dispatching.

Config D: Without RL-QT | Mean Wait: 880.6 ± 584.0 ms | +8.4% Latency Loss
Replaced RL-QT with static q=16 ms. Quantum fails to adapt to burst dispersion, triggering redundant preemptions.

Config C: Without Predictor | Mean Wait: 914.7 ± 601.8 ms | +12.6% Latency Loss
Replaced neural MLP with linear EWMA. Inability to capture heavy-tailed bursts severely degrades dispatch ordering.

Config B: Without Dispatch Agent | Mean Wait: 901.7 ± 564.9 ms | +11.0% Latency Loss
Used FIFO queue ordering with RL-QT. Interactive tasks suffer convoy delays behind compute-bound jobs.

Config A: Baseline System | Mean Wait: 947.5 ± 608.7 ms | +16.6% Latency Loss
Static q=16 ms + FIFO queue. Worst-case latency, preemption thrashing, and high energy consumption.

Conclusion: Neither quantum tuning alone (+11.0% loss) nor dispatch ordering alone (+8.4% loss) solves the dilemma. Dual-agent co-optimization is strictly mandatory.

EMPIRICAL VALIDATION

Statistical Significance & Line-Rate eBPF Kernel Verification

Wilcoxon Signed-Rank Test Results (N = 50 Seeds)

✓ vs Static Round Robin (q=16 ms)

p = 4.49 × 10^-9 (Statistically Significant)
Effect Size: r = 0.734 (Large effect)
Median Diff: +109.07 ms

✓ vs DABRR Heuristic (Hasan et al. 2024)

p = 1.78 × 10^-15 (Overwhelming Significance)
Effect Size: r = 0.869 (Very Large effect)
Median Diff: +150.32 ms

✓ vs Theoretical RR-Oracle-q

p = 8.75 × 10^-7 (Statistically Significant)
Effect Size: r = 0.634 (Large effect)
Median Diff: +59.18 ms

✓ Scientific Takeaway

Every p-value is far below 0.01; the null hypothesis is conclusively rejected. QTick surpasses even theoretical oracle quantum tuning by co-optimizing dispatch ordering.

Production Linux eBPF / sched_ext Deployment

In-Kernel eBPF Architecture

Implemented using the Linux kernel sched_ext framework (Kernel 6.6+). Attaches safely to kernel dispatch hooks without kernel recompilation.

Lock-Free Ring Buffers

BPF ring buffers communicate state vectors to the userspace inference daemon with zero spinlock lock-contention.

Line-Rate Dispatch Latency: 0.42 µs

Empirically measured dispatch hook execution time is 0.42 µs per scheduling event, well below standard 1.0 µs hardware timer boundaries.

Safe Verification by eBPF Verifier

100% verified by the kernel eBPF bytecode verifier: memory-safe, crash-proof, and work-conserving.

ORAL REVIEW CHEAT SHEET

Viva Voce Defense Guide: Anticipated Faculty Questions

Q1: How do you know task burst times without cheating (oracle knowledge)?

Answer: We never use future knowledge. We track the last 4 execution bursts in an online MLP predictor (MAE 4.12 ms) with exponential decay, exactly like CPU branch predictors and TCP RTT estimators. Even with prediction noise, QTick outperforms static schedulers by double digits.

Q2: Isn't neural network inference too heavy for an operating system kernel?

Answer: QTick decouples quantum tuning into a tiny 4,998-parameter MLP (19.5 KB) and tabular lookup. Total footprint is <60 KB, fitting entirely inside the CPU's 64 KB L1 cache. Inference executes in 0.18 µs on AVX2, and total dispatch takes 0.42 µs—well below kernel limits.

Q3: Why not just use Linux CFS or EEVDF?

Answer: CFS and EEVDF rely on static sysctl knobs (min_granularity_ns) that cannot dynamically adapt to cloud traffic. Under real bursty workloads, CFS suffers from 3,050 context switches, repeatedly evicting cache lines and degrading CPI by 350 µs per switch. QTick cuts switches to 53.

Q4: What happens if the RL agent suggests an unsafe or starving action?

Answer: The Lyapunov safety kernel enforces strict mathematical guardrails. If any task's queue waiting age exceeds tau_starve (50 ms), the safety kernel forcibly overrides the RL policy and grants immediate execution priority, mathematically guaranteeing starvation-freedom.

RESEARCH SUMMARY & FUTURE ROADMAP

Conclusion: QTick Proves RL Practicality in OS Kernels

★ Resolves the Quantum Dilemma: Decoupled multi-agent architecture co-optimizes time quantum length and queue dispatching without exponential state explosion.

★ 57.3% Latency Drop & 98.3% Switch Slash: Conclusively outperforms Linux CFS, standard Round Robin, and 2023–2025 SOTA models on 10,000+ real production tasks.

★ L1 Cache Residency & Sub-Microsecond Speed: <60 KB working footprint and 0.42 μs eBPF dispatch latency confirm real-world Linux kernel deployability.

★ Future Scope: (1) ARM big.LITTLE heterogeneous core asymmetry; (2) Multi-socket NUMA remote interconnect optimization; (3) Hard real-time automotive deadlines.

Thank you! Open for Questions & Technical Discussion.